

```
server.send(200, "application/json",
    "{\"success\":true,\"mode\":\"" + String(mode) + "\"}");
}
```

ESP32 16-Channel Relay Smart Switch — Full Feature Documentation

text

```
=====
*   ESP32 16-Channel Relay Smart Switch
*   Author: Raff Alds
*   Github: https://www.github.com/xiv3r
*   Project: Home, Business, Farm Automation etc...
=====
```

Table of Contents

1. Overview
2. Hardware Specifications
3. Architecture
4. Core Features
5. Time Management System
6. Schedule Engine
7. Web Interface
8. API Reference
9. Self-Healing System
10. GPIO Configuration
11. WiFi & Network Management
12. NTP & Time Synchronization
13. Access Point Configuration
14. System Monitoring & Recovery

- 15. Factory Reset Methods
- 16. Configuration Storage
- 17. Data Structures
- 18. Constants & Timing
- 19. Pin Mappings

Overview

The ESP32 16-Channel Relay Smart Switch is a comprehensive IoT automation platform designed for controlling up to 16 relays via schedules, manual overrides, and a responsive web interface. It features robust time synchronization (NTP, browser sync, DS3231 hardware RTC), self-healing capabilities, and persistent configuration storage.

Attribute	Value
Microcontroller	ESP32 (38-pin variant)
Max Relays	16
Schedules per Relay	8
Web Server	HTTP on port 80
Configuration Storage	NVS (Preferences)
RTC	DS3231 (I ² C) + Internal millis-based
WiFi Modes	STA + AP simultaneously
mDNS	Supported (.local domain)

Hardware Specifications

Supported Components

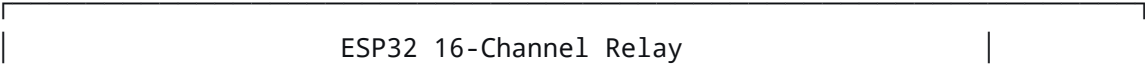
Component	Interface	Pins	Notes
ESP32	—	—	38-pin variant
DS3231 RTC	I ² C	GPIO21 (SDA), GPIO22 (SCL)	Optional but recommended
16-Channel Relay Module	GPIO	Configurable (see pin mapping)	Active LOW by default
BOOT Button	GPIO0	Pull-up	Factory reset trigger

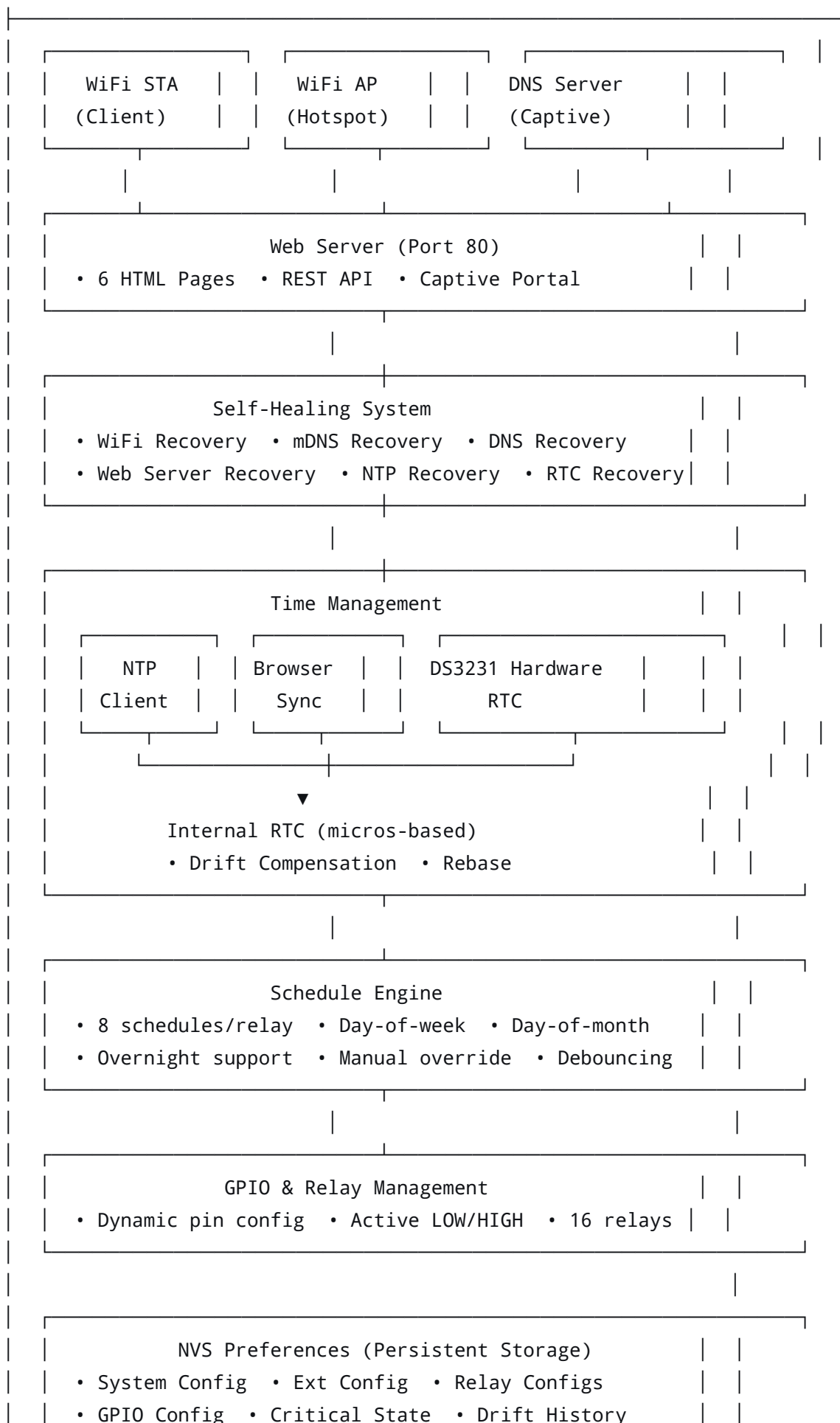
Default Relay GPIO Mapping

Relay	GPIO Pin	Relay	GPIO Pin
IN1	15	IN9	23
IN2	2	IN10	13
IN3	4	IN11	14
IN4	5	IN12	27
IN5	18	IN13	26
IN6	19	IN14	25
IN7	3	IN15	33
IN8	1	IN16	32

Architecture

text





Core Features

Feature Matrix

Feature	Description	Configurable
Relay Control	Manual ON/OFF/Auto per relay	Yes
Scheduling	8 independent schedules per relay	Yes
Day Selection	Individual days, weekdays, weekends, everyday	Yes
Month Day Selection	Specific days of month (1–31)	Yes
Overnight Scheduling	Cross-midnight schedules supported	Yes
Always-ON Detection	Start=Stop time = always ON	Automatic
Active Level	Per-relay LOW/HIGH or global mode	Yes
Relay Naming	Custom names up to 15 characters	Yes
GPIO Remapping	Dynamic pin reassignment	Yes
Web Dashboard	Responsive 6-page web interface	N/A
REST API	Full JSON API for all functions	N/A
Captive Portal	Automatic redirect for WiFi setup	Yes
mDNS	.local domain name resolution	Yes
NTP Sync	Multi-server failover (4 servers)	Yes

Feature	Description	Configurable
Browser Time Sync	Fallback time from browser	Manual
DS3231 RTC	Battery-backed hardware clock	Auto-detect
Drift Compensation	Automatic crystal drift correction	Automatic
Self-Healing	Automatic service recovery	Automatic
Factory Reset	Boot button (5s hold) or Web UI	Yes
Persistent Storage	NVS with magic number validation	Automatic
Memory Management	Periodic cleanup, heap monitoring	Automatic
Critical State Backup	Relay states preserved across crashes	Automatic

Time Management System

Time Sources (Priority Order)

Priority	Source	Accuracy	Persistence
1	NTP	±10ms (typical)	No
2	DS3231 Hardware RTC	±2ppm (±1 min/year)	Yes (battery)
3	Browser Sync	±1 second	No
4	Internal RTC (NVS)	±20ppm with drift comp	Yes (flash)

Time Source Indicators

Web UI Color	Source	Description
 Green	NTP	High accuracy, WiFi-connected
 Blue	Browser	Moderate accuracy
 Purple	RTC	DS3231 hardware clock
 Yellow	None/Internal	No time source available

Drift Compensation Algorithm

text

$$\text{driftCompensation} = \text{driftCompensation} \times 0.80 + \text{measuredRate} \times 0.20$$

Range: 0.95 - 1.05

History: 4-sample rolling average

DS3231 RTC Integration

Feature	Implementation
Interface	I ² C on GPIO21/22
Auto-detection	On boot, checks for device
Power-loss detection	<code>rtc.lostPower()</code> check
Sync interval	Every 60 seconds from internal RTC
Immediate sync	After NTP/browser time acquisition

Schedule Engine

Schedule Structure

text

Each Relay has 8 independent schedules:

Schedule N			
Start Time	Stop Time	Conditions	
(HH:MM:SS)	(HH:MM:SS)		
Enabled	Days (bit)	Month Days (bitmask)	
true/false	0x01=Sun	0x00000001 = Day 1	
	0x02=Mon	0x80000000 = Day 31	
	0x7F=All	0xFFFFFFFF = All	

Day Bitmask Constants

Constant	Value	Days
DAY_SUNDAY	0x01	Sunday only
DAY_MONDAY	0x02	Monday only
DAY_TUESDAY	0x04	Tuesday only
DAY_WEDNESDAY	0x08	Wednesday only
DAY_THURSDAY	0x10	Thursday only
DAY_FRIDAY	0x20	Friday only
DAY_SATURDAY	0x40	Saturday only
DAY_ALL	0x7F	Every day
DAY_WEEKDAYS	0x3E	Mon–Fri
DAY_WEEKENDS	0x41	Sun + Sat

Schedule Evaluation Logic

text

- For each relay:
- 1. If manual override → use manual state
 - 2. For each enabled schedule:
 - a. Check day-of-week mask
 - b. Check day-of-month mask (if non-zero)
 - c. Check time window:
 - start == stop → Always ON
 - start < stop → Normal range
 - start > stop → Overnight (crosses midnight)
 - 3. State debounced (500ms) before relay change
 - 4. Updates at 250ms interval

Special Schedule Types

Type	Condition	Web UI Badge
Always ON	Start == Stop	 Always ON
Overnight	Start > Stop	 Overnight
Day-restricted	Days != ALL	Shows day names
Month-restricted	Month days set	Shows day numbers

Web Interface

Page Structure

Page	Route	Description
Relays	/	Main dashboard with relay controls & schedules

Page	Route	Description
WiFi	/wifi	Station mode settings, network scanner
Time	/ntp	NTP configuration, manual sync buttons
AP	/ap	Access point settings
GPIO	/gpio	Pin configuration, active level settings
System	/system	System info, uptime, factory reset

Shared Header

text

[Relays] [WiFi] [Time] [AP] [GPIO] [System]		12:34:56
---	-----------------------------------	----------

Element	Description
Nav links	Current page highlighted
WiFi dot	Green = connected, Red = disconnected
Time dot	Green=NTP, Blue=Browser/RTC, Yellow=None
Clock	Local time (HH:MM:SS)

UI Color Scheme

Color	Usage
#1565C0	Primary blue (headers, buttons)
#0D47A1	Dark blue (gradient)

Color	Usage
#43A047	Green (ON state)
#E53935	Red (OFF state)
#546E7A	Gray (Auto mode)
#2E7D32	Success
#C62828	Danger/Error
#F9A825	Warning
#7B1FA2	Month-day selection

Relay Card Layout

text

Relay Name (dbl-click to edit)	MANUAL
[ON] [OFF] [AUTO]	
☒ Sched 1  Overnight (...)	
Start: [12:00:00] Stop: [08:00:00]	
Sun [✓] Mon [✓] Tue [✓] Wed [] ...	
1[✓] 2[✓] 3[] ... 30[] 31[]	
☐ Sched 2	
...	
[📁 Save Relay Name]	

Toast Notification System

Type	CSS Class	Timing
Success	ok (#2E7D32)	3 seconds
Error	er (#C62828)	3 seconds

Responsive Breakpoints

Breakpoint	Behavior
> 500px	Grid: auto-fill columns (340px min)
≤ 500px	Single column, compact inputs

API Reference

Base URL

text

http://<esp32-ip>/

http://<hostname>.local/

Endpoints

Relay Endpoints

Method	Endpoint	Description
GET	/api/relays	Get all relay states and schedules
POST	/api/relay/manual	Set manual ON/OFF {"relay":0,"state":true}
POST	/api/relay/reset	Return to auto mode {"relay":0}

Method	Endpoint	Description
POST	/api/relay/save	Save schedules {"relay":0,"schedules":[...]}
POST	/api/relay/name	Rename relay {"relay":0,"name":"Pump"}

Time Endpoints

Method	Endpoint	Description
GET	/api/time	Get current time, time source, WiFi status
POST	/api/time/browser-sync	Sync time from browser {"utc_epoch":1234567890}

WiFi Endpoints

Method	Endpoint	Description
GET	/api/wifi	Get WiFi status, SSID, IP, RSSI
POST	/api/wifi	Save WiFi credentials {"ssid":"...","password":"..."}
POST	/api/wifi/scan	Start network scan
GET	/api/wifi/scan	Get scan results

NTP Endpoints

Method	Endpoint	Description
GET	/api/ntp	Get NTP settings

Method	Endpoint	Description
POST	/api/ntp	Save NTP settings {"ntpServer":"...", "gmtOffset":28800, "daylightOffset":0, "syncHours":1}
POST	/api/ntp/sync	Force NTP synchronization

AP Endpoints

Method	Endpoint	Description
GET	/api/ap	Get AP settings
POST	/api/ap	Save AP settings {"ap_ssid":"...", "ap_password":"...", "ap_channel":6, "ap_hidden":false}

GPIO Endpoints

Method	Endpoint	Description
GET	/api/gpio	Get GPIO configuration
POST	/api/gpio/save	Save pin mapping {"pins":[15,2,4,...]}
POST	/api/gpio/add	Add single relay pin {"pin":27}
POST	/api/gpio/delete	Remove relay {"index":0}
POST	/api/gpio/toggle-active-low	Toggle active level {"index":0}
GET	/api/gpio/global-mode	Get global active mode
POST	/api/gpio/global-mode	Set global mode {"mode":0}

System Endpoints

Method	Endpoint	Description
GET	/api/system	Get system info (IP, uptime, heap, etc.)
POST	/api/reset	Soft reset / verify services
POST	/api/factory-reset	Factory reset all settings
GET	/api/mdns	Get mDNS info
POST	/api/mdns	Set mDNS hostname { "hostname": "esp32" }
POST	/api/mdns/restart	Restart mDNS service

Captive Portal Endpoints

Endpoint	Purpose
/generate_204	Android connectivity check
/hotspot-detect.html	Apple captive portal
/library/test/success.html	Apple captive portal
/success.txt	Generic success
/canonical.html	Redirect
/connecttest.txt	Windows connectivity check
/ncsi.txt	Windows NCSI
/redirect	Generic redirect

Response Format Example

GET /api/relays

json

```
[
  {
    "state": false,
    "manual": false,
    "name": "Relay 1",
    "pin": 15,
    "schedules": [
      {
        "startHour": 12,
        "startMinute": 0,
        "startSecond": 0,
        "stopHour": 8,
        "stopMinute": 0,
        "stopSecond": 0,
        "enabled": true,
        "days": 127,
        "monthDays": 0
      }
    ]
  }
]
```

GET /api/time

json

```
{
  "time": "14:30:25",
  "wifi": true,
  "ntp": true,
  "timeSource": "ntp",
  "rtcPresent": true
}
```

GET /api/system

json

```
{
  "ip": "192.168.1.100",
  "ap_ip": "192.168.4.1",
}
```



```
"uptime": 3600,
"freeHeap": 200000,
"utcEpoch": 1700000000,
"timeSource": "NTP",
"ntpServer": "time.google.com",
"ntpSyncAge": 45,
"browserSyncAge": -1,
"wifiConnected": true,
"wifiSSID": "MyNetwork",
"rssi": -55,
"version": 9,
"chipModel": "ESP32-38P",
"mdnsHostname": "esp32",
"mdnsStarted": true,
"relayCount": 16,
"maxRelays": 16,
"gmtOffset": 28800,
"driftComp": 1.0003,
"globalActiveMode": 0,
"rtcPresent": true
}
```

Self-Healing System

Health Metrics Tracking

Metric	Description	Recovery Action
wifiFailures	WiFi disconnection count	Force reconnect after 3 failures
ntpFailures	NTP sync failure count	Server rotation, retry
mdnsFailures	mDNS service failures	Service restart
dnsFailures	DNS failures	Service restart
webServerFailures	Web server failures	Live reconfiguration

Recovery Functions

Function	Trigger	Action
recoverWiFi()	WiFi disconnected	Re-begin with stored credentials
recoverMDNS()	mDNS not responding	MDNS.begin() restart
recoverDNS()	DNS issues	DNS server restart
recoverWebServer()	Web unresponsive	Server reconfiguration
recoverNTP()	NTP sync failed	Multi-server failover attempt
recoverRTC()	RTC lost	Reinitialize I ² C, reload epoch
verifyRelayStates()	30s interval	Read pin, correct if mismatch
performTargetedRecovery()	On-demand/5s boot	Full service chain restart

Critical State Preservation

text

CriticalRelayState (NVS)	
Magic: 0xDEADBEEF	
States: [16 bool]	
Overrides: [16 bool]	
Timestamp: uint32	
Checksum: uint32	
Saved: Every 5 minutes	
Restored: On boot (if valid)	
Validated: Magic + Checksum	

GPIO Configuration

Active Level Modes

Mode	Value	Description
Per-Relay	0	Each relay configured individually
Global LOW	1	All relays active LOW (digitalWrite inverted)
Global HIGH	2	All relays active HIGH (direct digitalWrite)

GPIO Configuration Storage

```
c
struct GPIOPinConfig {
    uint8_t pins[MAX_RELAYS]; // GPIO pin numbers
    uint8_t count;             // Active relay count
    uint16_t magic;            // 0xD002 validation
    bool activeLow[MAX_RELAYS]; // Per-relay active level
};
```

Dynamic GPIO Management

Operation	API	Behavior
Add	/api/gpio/add	Appends new relay at end
Delete	/api/gpio/delete	Removes relay, shifts remaining
Save All	/api/gpio/save	Full pin remapping
Toggle Level	/api/gpio/toggle-active-low	Flips active LOW/HIGH for one relay
Global Mode	/api/gpio/global-mode	Sets mode 0/1/2 for all relays

WiFi & Network Management

WiFi Modes

Mode	Behavior
WIFI_AP_STA	Simultaneous AP + STA (always active)

Connection Management

Parameter	Value
Reconnect attempts	10 (then 5-min cooldown)
Connection timeout	20 seconds
WiFi check interval	10 seconds
First attempt	Treated specially (no cooldown)

Network Scanner

Feature	Detail
Scan timeout	10 seconds
Max results	30 networks
Async scan	Non-blocking
Result display	Sorted by RSSI (strongest first)
Quick-select	Click network to fill SSID

mDNS Configuration

Setting	Default	Range
Hostname	esp32 (or AP SSID derived)	1–31 chars
Service	http._tcp on port 80	—
TXT records	model, version, channels	—

Captive Portal

Feature	Implementation
DNS redirection	All domains → AP IP
HTTP redirect	302 to dashboard
OS support	Android, Apple, Windows

NTP & Time Synchronization

NTP Server Pool

Index	Server	Fallback Order
0	time.google.com	Primary
1	time.windows.com	Fallback 1
2	time.cloudflare.com	Fallback 2
3	time.facebook.com	Fallback 3

Sync Configuration

Parameter	Default	Range
Sync interval	1 hour	1–24 hours
GMT offset	28800 (UTC+8)	Any int32
DST offset	0	Any int32
Retry interval	30 seconds	Fixed

Time Source Tracking

c

```
#define TIME_SOURCE_NONE    0    // No valid time
#define TIME_SOURCE_NTP     1    // NTP synchronized
#define TIME_SOURCE_BROWSER 2    // Browser sync fallback
#define TIME_SOURCE_RTC     3    // DS3231 hardware RTC
```

Browser Time Sync

text

```
POST /api/time/browser-sync
Body: {"utc_epoch": 17000000000}
```

Response:

```
{
  "success": true,
  "utc_epoch": 17000000000,
  "local_time": "2023-11-15 14:30:00",
  "gmt_offset": 28800,
  "time_source": "browser",
  "rtc_present": true,
  "drift": 1.0003
}
```

Access Point Configuration

Default Settings

Setting	Default Value
SSID	ESP32_16CH_Timer_Switch
Password	ESP32-admin
Channel	6
Hidden	No (SSID broadcast)
IP	192.168.4.1

Configuration Options

Parameter	Range	Description
SSID	1–31 chars	AP network name
Password	8–31 chars or empty (open)	WPA2 key
Channel	1–13	WiFi channel
Hidden	true/false	SSID broadcast

System Monitoring & Recovery

Memory Management

Metric	Threshold	Action
Heap check	Every 5 min	Track min free heap
Low heap	< 20KB	Full memory cleanup

Metric	Threshold	Action
Periodic cleanup	Every 30 min	Scan delete, NVS refresh, malloc/free cycle
Response cache	5s expiry	Clear stale cached JSON

Connection Monitoring

Metric	Interval	Action
Connection activity	5 min idle	Web server reconfiguration
Stale connections	60s	Close orphaned clients (max 5)

Schedule Processing

Interval	Function
250ms	Schedule evaluation
500ms	Relay state debounce
1000ms	Schedule cache update
300s	RTC rebase (drift compensation)
3600s	Full RTC state save

Factory Reset Methods

Method 1: Boot Button (Hardware)

Step	Description
1	Hold GPIO0 (BOOT) button
2	Wait 5 seconds
3	All settings cleared to defaults
4	WiFi reconnects if STA configured
5	AP restarts with default SSID

Method 2: Web UI (Software)

Step	Description
1	Navigate to <code>/system</code> page
2	Click "Factory Reset" button
3	Confirm dialog
4	All NVS cleared, defaults loaded
5	Page redirects to dashboard after 7s

Method 3: API

text

POST `/api/factory-reset`

Response: `{"success":true,"message":"Factory reset complete..."}`

Configuration Storage

NVS Namespace

text

Namespace: "relay16"

Stored Data Blobs

Key	Structure	Size	Magic
sysConfig	SystemConfig	~128 bytes	0x1234
relayConfigs	RelayConfig[MAX_RELAYS]	~3.5KB	—
extConfig	ExtConfig	32 bytes	0xEC
gpioConfig	GPIOPinConfig	~50 bytes	0xD002
criticalState	CriticalRelayState	~50 bytes	0xDEADBEEF

System Configuration

```
c

struct SystemConfig {
    uint16_t magic;           // 0x1234
    uint8_t  version;        // 9
    char     sta_ssid[32];
    char     sta_password[64];
    char     ap_ssid[32];
    char     ap_password[32];
    char     ntp_server[48];
    int32_t  gmt_offset;      // Seconds (e.g., 28800 = UTC+8)
    int32_t  daylight_offset; // Seconds DST
    time_t   last_rtc_epoch;  // Last saved epoch
    float    rtc_drift;       // Drift compensation factor
    char     hostname[32];    // mDNS hostname
};
```

Extended Configuration

c

```
struct ExtConfig {
    uint8_t magic;           // 0xEC
    uint8_t ap_channel;      // 1-13, default 6
    uint8_t ntp_sync_hours;  // 1-24, default 1
    uint8_t ap_hidden;       // 0=visible, 1=hidden
    uint8_t global_active_mode; // 0=per-relay, 1=all LOW, 2=all HIGH
    uint8_t reserved[27];
};
```

Relay Configuration

c

```
struct RelayConfig {
    TimerSchedule schedule;    // 8 schedules
    bool          manualOverride;
    bool          manualState;
    char          name[16];
};

struct TimerSchedule {
    uint8_t startHour[8], startMinute[8], startSecond[8];
    uint8_t stopHour[8], stopMinute[8], stopSecond[8];
    bool    enabled[8];
    uint8_t days[8];           // Day-of-week bitmask
    uint32_t monthDays[8];     // Day-of-month bitmask (1<=0 = day 1)
};
```

Data Structures

Health Metrics

c

```
struct HealthMetrics {
    uint32_t wifiFailures;
    uint32_t ntpFailures;
```

```

uint32_t mdnsFailures;
uint32_t dnsFailures;
uint32_t webServerFailures;
uint32_t lastRecoveryAttempt;
bool inRecoveryMode;
};

```

Response Cache

```

c

struct ResponseCache {
    String relaysJson;
    String systemJson;
    String timeJson;
    unsigned long lastUpdate;
    bool valid;
};

```

Self-Healing System Class

```

c

class SelfHealingSystem {
public:
    // Live reconfiguration
    bool liveReconfigureWiFi();
    bool liveReconfigureMDNS();
    bool liveReconfigureDNS();
    bool liveReconfigureWebServer();
    bool liveReconfigureAP();
    void restartAPIfNeeded(bool forceRestart);

    // Recovery
    bool recoverWiFi();
    bool recoverMDNS();
    bool recoverDNS();
    bool recoverWebServer();
    bool recoverNTP();
    bool recoverRTC();
    void smartRecovery();
};

```

```

void performTargetedRecovery();

// State preservation
void verifyRelayStates();
void saveCriticalState();
bool restoreCriticalState();

private:
    unsigned long lastMDNSAnnounce;
    unsigned long lastWebServerCheck;
    unsigned long lastFullHealthCheck;
    unsigned long lastWiFiReconfigure;
    unsigned long lastMDNSReconfigure;
    unsigned long lastDNSReconfigure;
};

```

Constants & Timing

EEPROM / NVS Constants

Constant	Value	Description
NVS_NAMESPACE	"relay16"	Preferences namespace
EEPROM_MAGIC	0x1234	System config validation
EEPROM_VERSION	9	Config structure version
EXT_CFG_MAGIC	0xEC	Extended config validation
GPIO_CONFIG_MAGIC	0xD002	GPIO config validation

Day Constants

Constant	Value	Binary
DAY_SUNDAY	0x01	0000001

Constant	Value	Binary
DAY_MONDAY	0x02	0000010
DAY_TUESDAY	0x04	0000100
DAY_WEDNESDAY	0x08	0001000
DAY_THURSDAY	0x10	0010000
DAY_FRIDAY	0x20	0100000
DAY_SATURDAY	0x40	1000000
DAY_ALL	0x7F	1111111
DAY_WEEKDAYS	0x3E	0111110
DAY_WEEKENDS	0x41	1000001

Timing Constants (milliseconds)

Constant	Value	Description
NTP_RETRY_INTERVAL	30,000	NTP retry delay
WIFI_CHECK_INTERVAL	10,000	WiFi status check
WIFI_CONNECT_TIMEOUT	20,000	Connection timeout
RTC_UPDATE_INTERVAL	100	RTC micros check
SCHEDULE_PROCESS_INTERVAL	250	Schedule evaluation
RELAY_UPDATE_INTERVAL	500	Relay debounce
RTC_REBASE_INTERVAL	300,000	Drift compensation (5 min)
RTC_SYNC_INTERVAL	3,600,000	Full RTC save (1 hour)

Constant	Value	Description
MEMORY_CLEANUP_INTERVAL	30,000	Light cleanup (30 sec)
MEMORY_CHECK_INTERVAL	60,000	Heap check (1 min)
CONNECTION_TIMEOUT	10,000	Web client timeout
SCHEDULE_CACHE_INTERVAL	1,000	Cache refresh
FACTORY_RESET_HOLD	5,000	Boot button hold time
MDNS_RESTART_DELAY	2,000	mDNS restart delay

Max Reconnect Attempts

Constant	Value
MAX_RECONNECT	10
Cooldown period	300,000ms (5 min)

Pin Mappings

Valid GPIO Pins for Relays

text
GPIO: 15, 2, 4, 5, 18, 19, 3, 1, 23, 13, 14, 27, 26, 25, 33, 32

Special Pins

Pin	Function	Notes
GPIO0	BOOT button	Pull-up, factory reset trigger

Pin	Function	Notes
GPIO21	I ² C SDA	DS3231 RTC data
GPIO22	I ² C SCL	DS3231 RTC clock

Pin Safety Notes

GPIO	Notes
6–11	Connected to internal flash, avoid
0, 2, 5, 12, 15	Boot strapping pins, check states
34–39	Input only, cannot use for relays

License & Credits

text

```
=====
*   ESP32 16-Channel Relay Smart Switch
*   Author: Raff Alds
*   Github: https://www.github.com/xiv3r
*   Project: Home, Business, Farm Automation etc...
=====
```

Dependencies

Library	Purpose
WiFi.h	WiFi connectivity
NTPClient.h	NTP time synchronization
WiFiUdp.h	UDP for NTP